SRM

INSTITUTE OF SCIENCE & TECHNOLOGY

(Deemed to be **University** u/s 3 of UGC Act, 1956)

DELHI NCR CAMPUS, GHAZIABAD, (U.P.)

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(FACULTY OF SCIENCE & HUMANITIES)

DEPARTMENT OF COMPUTER APPLICATIONS

PRACTICAL FILE

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

[PCA25C07J]

MCA (Gen AI) 1ST YEAR, 2 SEMESTER

Session: 2025-26

SUBMITTED TO :

Ms. Beena Sachan
Assistant Professor
Department of Computer Applications

SUBMITTED BY :

Student Reg No: RA2532242030041
Student Name: ADITYA SINGH
Department of Computer Applications

S R M INSTITUTE OF SCIENCE & TECHNOLOGY

DELHI NCR CAMPUS, MODINAGAR

(FACULTY OF SCIENCE & HUMANITIES)

DEPARTMENT OF COMPUTER APPLICATIONS

Register No. : **RA2532242030041**

BONAFIDE CERTIFICATE

Certified to be the Bonafide record of the work done by **ADITYA SINGH** of **MCA (Gen AI) 1ST Year/ 2 Semester** for the Award of **Master's Degree** in the **DEPARTMENT OF COMPUTER APPLICATIONS** in **Artificial Intelligence And Machine Learning [PCA25C07J]** laboratory during the Academic Year **2025-26.**

SUBJECT IN – CHARGE **HEAD – COMPUTER APPLICATIONS**

Submitted for the University Examination held on _____.

INTERNAL EXAMINER - 1 **INTERNAL EXAMINER – 2**

Date:…./……../……

Experiment – 1

Experiment 1: Solving Problems using AI: Vacuum Cleaner World (Simple Reflex Agent)

Aim : To implement a **Simple Reflex Agent** for the **Vacuum Cleaner World problem** that cleans a room based on the current percept (location and dirt status).

Procedure :

- Define the environment with two locations **A** and **B**.
- Each location can be either **Clean** or **Dirty**.
- The agent perceives:
- Current **location**
- **Status** (Clean/Dirty).
- Apply simple reflex rules:
- If the location is **Dirty → Suck**.
- If **Clean and at A → Move Right**.
- If **Clean and at B → Move Left**.
- Continue until both locations are clean.

Source Code :

```
# Vacuum Cleaner World - Simple Reflex Agent


def vacuum_agent(location, status):

    if status == "Dirty":

        return "Suck"

    elif location == "A":

        return "Right"

    elif location == "B":

        return "Left"


# Environment

rooms = {

    "A": "Dirty",

    "B": "Dirty"

}location = "A"

for i in range(4):

    status = rooms[location]
```

```
    action = vacuum_agent(location, status)

    print("Location:", location)
    print("Status:", status)
    print("Action:", action)
    print()

    if action == "Suck":
        rooms[location] = "Clean"
    elif action == "Right":
        location = "B"
    elif action == "Left":
        location = "A"

print("Final Room Status:", rooms)
```

Output :

```
Location: A
Status: Dirty
Action: Suck
26 |
Location: A
Status: Clean
Action: Right
26 |
Location: B
Status: Dirty
Action: Suck
26 |
Location: B
Status: Clean
Action: Left
26 |
Final Room Status: {'A': 'Clean', 'B': 'Clean'}
```

Date:…./……../……

Experiment – 2

Experiment 2: Solving Problems using AI: WAP to implement Tower of Hanoi.

Aim : To write a program to solve the **Tower of Hanoi problem** using recursion.

Procedure :

- Define the number of disks **n**.
- Use three rods: **Source (A)**, **Auxiliary (B)**, and **Destination (C)**.
- Apply recursion with the following rules:
  - Move **n−1 disks** from Source to Auxiliary using Destination.
  - Move the **nth disk** from Source to Destination.
  - Move **n−1 disks** from Auxiliary to Destination using Source.
- Continue until all disks are moved to the destination rod.

Source Code :

```
# Tower of Hanoi using Recursion


def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    tower_of_hanoi(n-1, source, destination, auxiliary)
    print("Move disk", n, "from", source, "to", destination)
    tower_of_hanoi(n-1, auxiliary, source, destination)

# Number of disks
n = 3

tower_of_hanoi(n, 'A', 'B', 'C')
```

Output

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Date:……/……../……..

Experiment -3

Experiment 3: Propositional Logic and Reasoning: Write a Python program that generates the truth table of a propositional expression. (P->Q)

Aim : To write a Python program that generates the **truth table** for the propositional expression **(P → Q)**.

Procedure :

- Define the possible truth values **True** and **False** for propositions **P** and **Q**.
- Use nested loops to generate all combinations of **P** and **Q**.
- Implement the implication rule:
  - **P → Q is False only when P is True and Q is False**.
- Print the truth table showing **P, Q, and (P → Q)**.

Source Code :

```
# Truth Table for (P -> Q)

values = [True, False]

print("P\tQ\tP->Q")

for P in values:
    for Q in values:
        implication = (not P) or Q
        print(P, "\t", Q, "\t", implication)
```

Output :

```
P    Q    P->Q
True      True     True
True      False    False
False     True     True
False     False    True
```

Date:……/……../……..

Experiment - 4

Experiment 4: Experts System in Prolog: Create a Prolog rule-based expert system to recommend veg or non-veg restaurants.

Aim: To create a **rule-based expert system in Prolog** that recommends **veg or non-veg restaurants** based on user preferences.

Procedure:

- Define facts about **restaurants** and their types (veg or non-veg).
- Create rules to recommend restaurants based on the user's preference.
- Ask the user whether they prefer **veg or non-veg food**.
- The expert system checks the rules and suggests suitable restaurants.

Source Code:

```
% Facts: Restaurant types
restaurant(green_leaf, veg).
restaurant(pure_taste, veg).
restaurant(spicy_grill, nonveg).
restaurant(meat_house, nonveg).

% Rule to recommend veg restaurant
recommend(Restaurant) :-
    restaurant(Restaurant, veg),
    write('Recommended Veg Restaurant: '),
    write(Restaurant).

% Rule to recommend non-veg restaurant
recommend_nonveg(Restaurant) :-
    restaurant(Restaurant, nonveg),
    write('Recommended Non-Veg Restaurant: '),
    write(Restaurant).
```

Output:

```
</> prolog

?- recommend(R).
Recommended Veg Restaurant: green_leaf

?- recommend_nonveg(R).
Recommended Non-Veg Restaurant: spicy_grill
```

Date:……/……../……..

Experiment - 5

Experimemt 5: Working on Uninformed Search : WAP to implement Breadth-First Search (BFS)

Aim: To write a Python program to implement **Breadth-First Search (BFS)** for traversing a graph.

Procedure:

- Represent the graph using an **adjacency list**.
- Create a **queue** to store nodes to be visited.
- Start from the **initial node** and mark it as visited.
- Visit all **adjacent nodes** of the current node and add them to the queue.
- Continue the process until all nodes are visited.

Source Code

```
# Breadth First Search (BFS)
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()
queue = deque()

def bfs(start):
```

```
    visited.add(start)
    queue.append(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

# Start BFS from node A
bfs('A')
```

Output:

```
A
B
C
D
E
F
```

Date:……/……../……..

Experiment - 6

Experiment 6: Working on Uninformed Search : WAP to implement Depth-First Search (DFS)

Aim: To write a Python program to implement **Depth-First Search (DFS)** for traversing a graph.

Procedure:

- Represent the graph using an **adjacency list**.
- Create a **visited set** to keep track of visited nodes.
- Start from the **initial node**.
- Visit the node and mark it as **visited**.
- Recursively visit all **unvisited adjacent nodes** until all nodes are explored.

Source Code:

```
# Depth First Search (DFS)

graph = {
'A': ['B', 'C'],
'B': ['D', 'E'],
'C': ['F'],
'D': [],
'E': ['F'],
'F': []
}

visited = set()

def dfs(node):
if node not in visited:
print(node)
visited.add(node)

for neighbour in graph[node]:
dfs(neighbour)

# Start DFS from node A
dfs('A'
```

Output :

```
A
B
C
D
E
F
```

Date:……/……../……..

Experiment - 7

Experiment 7: Working on Informed Search : WAP to demonstrate Greedy Best-First Search to find a path from a start node to a goal node using heuristic values.

Aim : To write a Python program to demonstrate **Greedy Best-First Search** to find a path from a **start node to a goal node** using **heuristic values**.

Procedure :

- Represent the graph using an **adjacency list**.
- Assign **heuristic values (h(n))** to each node that estimate the distance to the goal node.
- Use a **priority queue** to select the node with the **lowest heuristic value**.
- Start from the **initial node** and explore the node that appears closest to the goal.
- Continue the process until the **goal node is reached**.

Code :

```
# Greedy Best First Search

from queue import PriorityQueue

graph = {
'A': ['B', 'C'],
'B': ['D', 'E'],
'C': ['F'],
'D': [],
'E': ['F'],
'F': []
}


# Heuristic values
heuristic = {
'A': 6,
'B': 4,
'C': 3,
'D': 2,
'E': 1,
'F': 0
}
```

```
 def greedy_best_first_search(start, goal):
visited = set()
pq = PriorityQueue()

pq.put((heuristic[start], start))

while not pq.empty():
node = pq.get()[1]

if node not in visited:
print(node)
visited.add(node)

if node == goal:
print("Goal Reached!")
return

for neighbour in graph[node]:
if neighbour not in visited:
pq.put((heuristic[neighbour], neighbour))

# Start search
greedy_best_first_search('A', 'F')
```

Output :

```
A
C
F
Goal Reached!
```

Date:……/……../……..

Experiment - 8

Experiment 8: Working on Informed Search: WAP to demonstrate an A* Search to find the shortest path between two nodes in a graph using a heuristic function.

Aim : To write a Python program to demonstrate the **A* Search Algorithm** to find the **shortest path between two nodes** in a graph using a **heuristic function**.

Procedure :

- Represent the graph with **nodes and edge costs**.
- Define a **heuristic function h(n)** that estimates the cost from the current node to the goal node.
- Calculate the evaluation function:

f(n) = g(n) + h(n)

- **g(n)** = cost from start node to current node
- **h(n)** = estimated cost from current node to goal

- Use a **priority queue** to select the node with the lowest **f(n)** value.
- Continue exploring nodes until the **goal node is reached**.

Code :-

```
# A* Search Algorithm

from queue import PriorityQueue

graph = {
'A': {'B': 1, 'C': 3},
'B': {'D': 1, 'E': 5},
'C': {'F': 2},
'D': {},
'E': {'F': 1},
'F': {}
}

heuristic = {
'A': 6,
'B': 4,
'C': 2,
'D': 1,
'E': 1,
'F': 0
}
```

```
def a_star(start, goal):
pq = PriorityQueue()
pq.put((0, start))

g_cost = {start: 0}

while not pq.empty():
current = pq.get()[1]
print(current)

if current == goal:
print("Goal Reached!")
return

for neighbour, cost in graph[current].items():
new_cost = g_cost[current] + cost

if neighbour not in g_cost or new_cost < g_cost[neighbour]:
g_cost[neighbour] = new_cost
f_cost = new_cost + heuristic[neighbour]
pq.put((f_cost, neighbour))

# Run A*
a_star('A', 'F')
```

Output :

```
A
B
D
C
F
Goal Reached!
```

Date:……/……../……..

Experiment - 9

Experiment 9 : Working with Prolog: Design and implement a Prolog program to represent students, courses, and grades, and answer queries related to student performance.

Aim : To design and implement a **Prolog program** to represent **students, courses, and grades**, and answer queries related to **student performance**.

Procedure :

- Define facts representing **students**, **courses**, and their **grades**.
- Create rules to determine **student performance** based on grades.
- Use Prolog queries to retrieve information such as **student grades, course details, and performance status**.
- Execute queries to analyze **student results**.

Code :

```
% Facts: Students
student(rahul).
student(priya).
student(aman).

% Facts: Courses
course(ai).
course(dbms).
course(math).

% Facts: Grades (student, course, grade)
grade(rahul, ai, 85).
grade(rahul, dbms, 78).
grade(priya, ai, 92).
grade(priya, math, 88).
grade(aman, dbms, 60).
grade(aman, math, 55).

% Rule to check if a student passed
pass(Student, Course) :-
grade(Student, Course, Marks),
Marks >= 50
```

```
% Rule to check if a student has distinction
distinction(Student, Course) :-
grade(Student, Course, Marks),
Marks >= 85.
```

Output :

```prolog
?- grade(rahul, ai, Marks).
Marks = 85.

?- pass(aman, dbms).
true.

?- distinction(priya, ai).
true.
```

Date:……/……../……..

Experiment - 10

Experiment 10 : Working on Control Structures in Prolog: Design and implement a Prolog program to demonstrate the useof control structures such as cut (!), fail, and if–then–else.

Aim : To design and implement a **Prolog program** to demonstrate the use of **control structures such as Cut (!), Fail, and If–Then–Else**.

Procedure :

- Define facts representing **student marks**.
- Use **cut (!) operator** to stop backtracking once a condition is satisfied.
- Use **fail** to force Prolog to backtrack and show alternative results.
- Implement **if–then–else (-> ;)** structure to check conditions and display results.
- Run queries to observe the behavior of these control structures.

Code :

```
% Facts: student marks
marks(rahul, 85).
marks(priya, 45).
marks(aman, 70).

% Demonstrating Cut (!)
result(Student, pass) :-
marks(Student, M),
M >= 50, !.

result(Student, fail) :-
marks(Student, M),
M < 50.

% Demonstrating Fail
show_student(Student) :-
marks(Student, M),
write(Student), write(' has marks '), write(M), nl,
fail.
```

```
show_student(_).

% Demonstrating If–Then–Else
check_pass(Student) :-
marks(Student, M),
(M >= 50 ->
write(Student), write(' passed')
;
write(Student), write(' failed')
).
```

Output :

```
</> prolog

?- result(rahul, R).
R = pass.

?- show_student(X).
rahul has marks 85
priya has marks 45
aman has marks 70

?- check_pass(priya).
priya failed
```

Date:……/……../……..

Experiment – 11

Experiment 11: Working on Recursion in Prolog: Design and implement a Prolog program to find the factorial of a number using recursion.

AIM : To design and implement a **Prolog program** to find the **factorial of a number using recursion**.

Procedure :

- Define a **base case** where the factorial of **0 is 1**.
- Define a **recursive rule** where factorial of **N** is calculated as:
  N × factorial(N−1).
- The recursion continues until the **base condition** is reached.
- Execute a query to calculate the factorial of a given number.

Code :

```
% Base case
factorial(0, 1).

% Recursive case
factorial(N, F) :-
N > 0,
N1 is N - 1,
factorial(N1, F1),
F is N * F1.
```

Output :

```
prolog

?- factorial(5, F).
F = 120.
```

Date:……/……../……..

Experiment – 12

Experiment 12: Supervised Learning: WAP to implement a supervised learning model using the Naïve Bayes algorithm to classify data.

AIM: To write a Python program to implement a **Supervised Learning model using the Naïve Bayes algorithm** to classify data.

Procedure :

- Import required libraries such as **NumPy** and **sklearn**.
- Prepare a **dataset** containing features and corresponding class labels.
- Split the dataset into **training and testing sets**.
- Train the model using the **Naïve Bayes classifier**.
- Use the trained model to **predict the class** of test data.
- Display the **prediction results and accuracy**.

Code :

```
# Naive Bayes Classification

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Train model
model = GaussianNB()
model.fit(X_train, y_train)
```

Output :

```
Predicted values: [1 0 0 1 2 2 2 2 1 2 1 1 1 1 1 1 2 1 0 0 0 2 1 1 2 0 1 2 0 1 1 2 2 0 2 0
 0 2 2 2 2 2 2 0]

Accuracy: 1.0
```

Experiment – 13

Experiment 13: Bayesian Learning: WAP to implement a Bayesian Learning model to predict whether a patient has a disease based on test results using Bayes' Theorem.

AIM: To write a Python program to implement a **Bayesian Learning model** to predict whether a **patient has a disease based on test results** using **Bayes' Theorem**.

.Procedure :

- Define the **prior probabilities** for the disease and no disease.
- Define the **likelihood probabilities** for positive and negative test results.
- Apply **Bayes' Theorem** to calculate the probability of having the disease given a positive test result.
- Compare the calculated probability to determine whether the patient is **likely to have the disease or not**.
- Display the result.

Code :

```
# Bayesian Learning using Bayes' Theorem
# Prior probabilities
P_disease = 0.01
P_no_disease = 0.99

# Likelihood probabilities
P_positive_given_disease = 0.9
P_positive_given_no_disease = 0.05

# Bayes Theorem
P_positive = (P_positive_given_disease * P_disease) + \
        (P_positive_given_no_disease * P_no_disease)

P_disease_given_positive = (P_positive_given_disease * P_disease) / P_positive
```

```
print("Probability of disease given positive test:", P_disease_given_positive)

if P_disease_given_positive > 0.5:
    print("Patient likely has the disease")
else:
    print("Patient unlikely to have the disease")

# Create button
button = tk.Button(root, text="Click Me", command=show_message)
```

Output :

```
Probability of disease given positive test: 0.15384615384615385

Patient unlikely to have the disease
```

Experiment – 14

Experiment 14: Linear Models for Clustering: WAP to implement a clustering model using a linear distance–based approach (K-Means) to group data into clusters.

AIM: To write a Python program to implement a **clustering model using the K-Means algorithm** to group data into clusters based on **distance between data points**.

.Procedure :

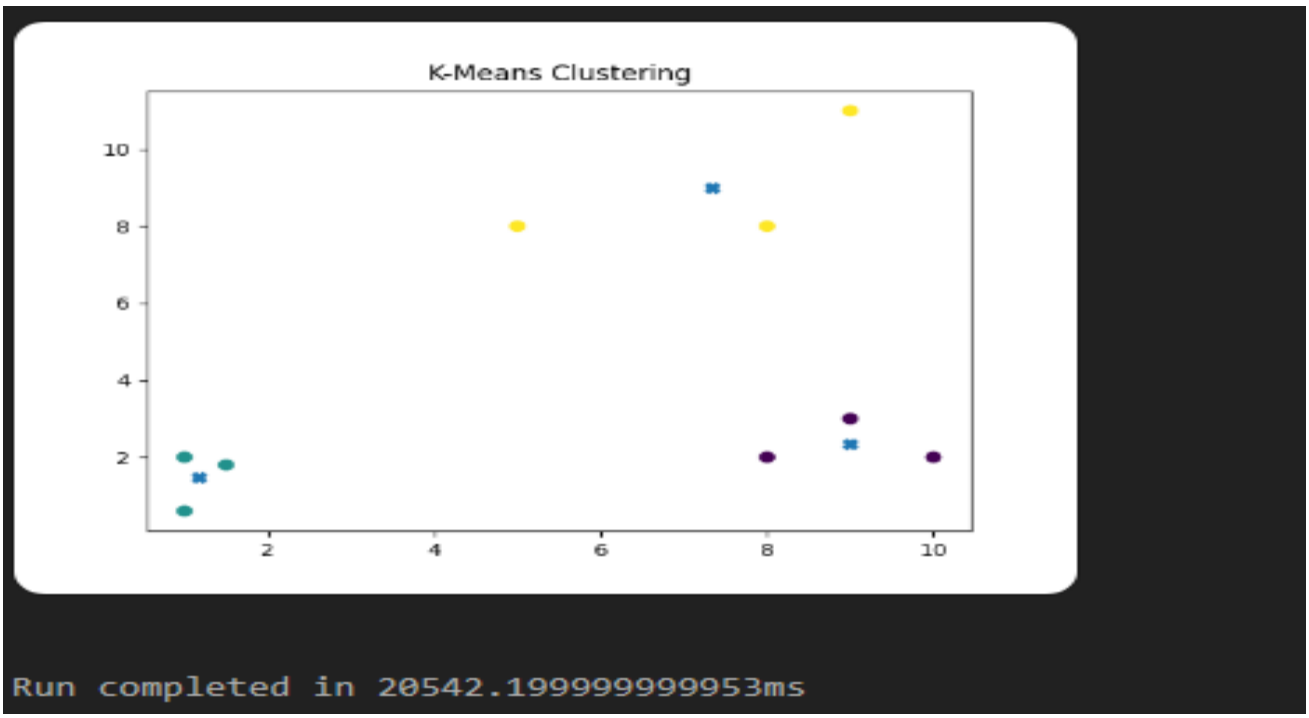
- Import required libraries such as **NumPy**, **Matplotlib**, and **sklearn**.
- Create or load a **dataset containing data points**.
- Choose the number of clusters **K**.
- Apply the **K-Means clustering algorithm** to group the data points.
- Plot the clusters and their **centroids** for visualization.

Code :

```
# K-Means Clustering
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans


# Sample dataset
X = np.array([
    [1, 2], [1.5, 1.8], [5, 8],
    [8, 8], [1, 0.6], [9, 11],
    [8, 2], [10, 2], [9, 3]
])


# Number of clusters
```

```
k = 3
# Apply K-Means
kmeans = KMeans(n_clusters=k)
kmeans.fit(X)

labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Plot clusters
plt.scatter(X[:,0], X[:,1], c=labels)
plt.scatter(centroids[:,0], centroids[:,1], marker='X')
plt.title("K-Means Clustering")
plt.show()
```

Output :



Date:……/……../……..

Experiment – 15

Experiment 15: Reinforcement Learning:WAP to implement a Reinforcement Learning model using Q-Learning to find the optimal path in a simple environment.

AIM: To write a Python program to implement a **Reinforcement Learning model using the Q-Learning algorithm** to find the **optimal path in a simple environment**.

Procedure :

- Define the **environment states and actions**.
- Initialize a **Q-table** with zeros for all state–action pairs.
- Set learning parameters such as **learning rate, discount factor, and episodes**.
- Update Q-values using the **Q-learning formula**:
Q(s,a)=Q(s,a)+α[r+γmax Q(s′,a′)−Q(s,a)]Q(s,a) = Q(s,a) + \alpha [r + \gamma \max Q(s',a') - Q(s,a)]Q(s,a)=Q(s,a)+α[r+γmaxQ(s′,a′)−Q(s,a)].
- Train the model through multiple **episodes** until the optimal policy is learned.
- Display the **optimal path or Q-table**.

Code :

```
import numpy as np


# Reward matrix
R = np.array([
    [-1, -1, -1, -1, 0, -1],
    [-1, -1, -1, 0, -1, 100],
    [-1, -1, -1, 0, -1, -1],
    [-1, 0, 0, -1, 0, -1],
    [0, -1, -1, 0, -1, 100],
    [-1, 0, -1, -1, 0, 100]
])
```

```
# Initialize Q-table
Q = np.zeros_like(R)

gamma = 0.8
episodes = 1000

for _ in range(episodes):
    state = np.random.randint(0, 6)

    while True:
        actions = np.where(R[state] >= 0)[0]
        action = np.random.choice(actions)

        next_state = action
        Q[state, action] = R[state, action] + gamma * np.max(Q[next_state])

        state = next_state

        if state == 5:
            break

print("Q-Table:")
print(Q)
```

Output :

```
Q-Table:
[[   0    0    0    0  396    0]
 [   0    0    0  316    0  496]
 [   0    0    0  316    0    0]
 [   0  396  252    0  396    0]
 [316    0    0  316    0  496]
 [   0  396    0    0  396  496]]
```